In [2]:
```
import pandas as pd

policy_df = pd.read_csv("policy_sales_data.csv", parse_dates=["Policy_Purchase_Date","Policy_Start_Date","Policy_End_Date"])
claims_df = pd.read_csv("claims_data.csv", parse_dates=["Claim_Date"])
```

In [3]:
```
policy_df.head()
```

Out[3]:

	Customer_ID	Vehicle_ID	Vehicle_Value	Policy_Tenure	Policy_Purchase_Date	Policy_Start_Date	Policy_End_Date	Premium
0	1	1000001	100000	3	2024-12-09	2025-12-09	2028-12-09	300
1	2	1000002	100000	3	2024-08-14	2025-08-14	2028-08-14	300
2	3	1000003	100000	3	2024-02-07	2025-02-06	2028-02-06	300
3	4	1000004	100000	2	2024-05-01	2025-05-01	2027-05-01	200
4	5	1000005	100000	1	2024-09-29	2025-09-29	2026-09-29	100

In [4]:
```
claims_df.head()
```

Out[4]:

	Claim_ID	Customer_ID	Vehicle_ID	Claim_Amount	Claim_Date	Claim_Type
0	1	97808	1097808	10000.0	2025-06-07	1
1	2	570945	1570945	10000.0	2025-07-07	1
2	3	320605	1320605	10000.0	2025-09-21	1
3	4	27619	1027619	10000.0	2025-09-28	1
4	5	999805	1999805	10000.0	2025-07-28	1

Q.1) Calculate the total premium collected during the year 2024.

In [5]:
```
total_premium = policy_df["Premium"].sum()
total_premium
```

Out[5]:
```
np.int64(2400000000)
```

Q.2) Calculate the total claim cost for each year (2025 and 2026) with a monthly breakdown.

In [6]:
```
claims_df.head()
```

Out[6]:

	Claim_ID	Customer_ID	Vehicle_ID	Claim_Amount	Claim_Date	Claim_Type
0	1	97808	1097808	10000.0	2025-06-07	1
1	2	570945	1570945	10000.0	2025-07-07	1
2	3	320605	1320605	10000.0	2025-09-21	1
3	4	27619	1027619	10000.0	2025-09-28	1
4	5	999805	1999805	10000.0	2025-07-28	1

In [7]:
```
claims_df["Year"] = claims_df["Claim_Date"].dt.year
claims_df["Month"] = claims_df["Claim_Date"].dt.month

claims_monthly = claims_df.groupby(["Year","Month"])["Claim_Amount"].sum().reset_index()
claims_monthly
```

Out[7]:

	Year	Month	Claim_Amount
0	2025	1	32240000.0
1	2025	2	32490000.0
2	2025	3	32320000.0
3	2025	4	32470000.0
4	2025	5	32820000.0
5	2025	6	31700000.0
6	2025	7	32300000.0
7	2025	8	33290000.0
8	2025	9	34000000.0
9	2025	10	32500000.0
10	2025	11	33450000.0
11	2025	12	32270000.0
12	2026	1	53000000.0
13	2026	2	47000000.0

Q.3) Calculate the claim cost to premium ratio for each policy tenure (1, 2, 3, and 4 years).

In [8]:
```
merged = claims_df.merge(policy_df, on=["Customer_ID","Vehicle_ID"], how="left")
merged.head()
```

Out[8]:

	Claim_ID	Customer_ID	Vehicle_ID	Claim_Amount	Claim_Date	Claim_Type	Year	Month	Vehicle_Value	Policy_Tenure	Policy_Purchase_Date	Policy_Start_Date	Policy_End_Date	Premium
0	1	97808	1097808	10000.0	2025-06-07	1	2025	6	100000	1	2024-06-07	2025-06-07	2026-06-07	100
1	2	570945	1570945	10000.0	2025-07-07	1	2025	7	100000	3	2024-07-07	2025-07-07	2028-07-07	300
2	3	320605	1320605	10000.0	2025-09-21	1	2025	9	100000	3	2024-09-21	2025-09-21	2028-09-21	300
3	4	27619	1027619	10000.0	2025-09-28	1	2025	9	100000	2	2024-09-28	2025-09-28	2027-09-28	200
4	5	999805	1999805	10000.0	2025-07-28	1	2025	7	100000	2	2024-07-28	2025-07-28	2027-07-28	200

In [9]:
```
ratio = merged.groupby("Policy_Tenure").agg(
    Total_Claims=("Claim_Amount","sum"),
    Total_Premium=("Premium","sum")
)
ratio["Claim_to_Premium_Ratio"] = ratio["Total_Claims"] / ratio["Total_Premium"]
ratio
```

Out[9]:

Policy_Tenure	Total_Claims	Total_Premium	Claim_to_Premium_Ratio
1	785900000.0	785900	100.000000
2	1173800000.0	2347600	50.000000
3	1562700000.0	4688100	33.333333
4	1396100000.0	5584400	25.000000

Q.4) Calculate the claim cost to premium ratio by the month in which the policy was sold (January–December 2024).

In [10]:
```
policy_df["Sale_Month"] = policy_df["Policy_Purchase_Date"].dt.month

merged = claims_df.merge(policy_df, on=["Customer_ID","Vehicle_ID"], how="left")

monthly_ratio = merged.groupby("Sale_Month").agg(
    Claims=("Claim_Amount","sum"),
    Premium=("Premium","sum")
)

monthly_ratio["Loss_Ratio"] = monthly_ratio["Claims"]/monthly_ratio["Premium"]
monthly_ratio
```

Out[10]:

Sale_Month	Claims	Premium	Loss_Ratio
1	41270000.0	1122900	36.753050
2	40350000.0	1097700	36.758677
3	40960000.0	1117100	36.676851
4	41040000.0	1126400	36.434659
5	41140000.0	1117400	36.817612
6	40330000.0	1106100	36.461441
7	40850000.0	1123500	36.359591
8	41780000.0	1142500	36.568928
9	41920000.0	1134400	36.953456
10	40640000.0	1106100	36.741705
11	41240000.0	1119300	36.844456
12	40430000.0	1092600	37.003478

Q.5) If every vehicle that has not yet made a claim eventually files exactly one claim during the remaining policy tenure, estimate the total potential claim liability.

In [11]:
```
claimed_vehicles = claims_df["Vehicle_ID"].nunique()

total_vehicles = policy_df["Vehicle_ID"].nunique()

remaining_vehicles = total_vehicles - claimed_vehicles

potential_liability = remaining_vehicles * 10000

potential_liability
```

Out[11]:
```
9512350000
```

Q.6)

• Calculate the premium already earned by the company up to February 28, 2026.

In [12]:
```
policy_df["Tenure_Days"] = policy_df["Policy_Tenure"] * 365

policy_df["Daily_Premium"] = policy_df["Premium"] / policy_df["Tenure_Days"]
```

In [13]:
```
cutoff = pd.Timestamp("2026-02-28")

policy_df["Earned_Days"] = (cutoff - policy_df["Policy_Start_Date"]).dt.days

policy_df["Earned_Days"] = policy_df["Earned_Days"].clip(lower=0)

policy_df["Earned_Days"] = policy_df[["Earned_Days","Tenure_Days"]].min(axis=1)

policy_df["Earned_Premium"] = policy_df["Earned_Days"] * policy_df["Daily_Premium"]

earned_premium = policy_df["Earned_Premium"].sum()

earned_premium
```

Out[13]:
```
np.float64(65888187.39726026)
```

• Estimate the premium expected to be earned monthly for the remaining policy period (assume 46 months remaining).

In [14]:
```
remaining_premium = policy_df["Premium"].sum() - earned_premium

monthly_expected = remaining_premium / 46

monthly_expected
```

Out[14]:
```
np.float64(3785039.4044073857)
```

Bonus Questions

Q.1) Identify which policy tenure appears most profitable and explain why

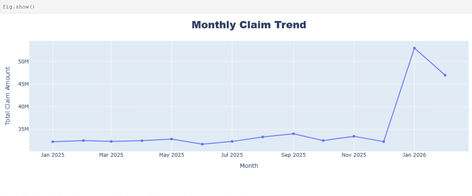
In [15]:
```
merged = claims_df.merge(policy_df, on=["Customer_ID","Vehicle_ID"], how="left")

tenure_analysis = merged.groupby("Policy_Tenure").agg(
    Total_Claims=("Claim_Amount","sum"),
    Total_Premium=("Premium","sum")
)

tenure_analysis["Loss_Ratio"] = tenure_analysis["Total_Claims"] / tenure_analysis["Total_Premium"]

tenure_analysis
```

Out[15]:

Policy_Tenure	Total_Claims	Total_Premium	Loss_Ratio
1	785900000.0	785900	100.000000
2	1173800000.0	2347600	50.000000
3	1562700000.0	4688100	33.333333
4	1396100000.0	5584400	25.000000

Q.2) Build a simple dashboard or visualization showing claim trends by month.

In [25]:
```
import plotly.express as px

# Create Month column
claims_df["Month"] = claims_df["Claim_Date"].dt.to_period("M").astype(str)

# Aggregate monthly claims
monthly_claims = claims_df.groupby("Month")["Claim_Amount"].sum().reset_index()

# Create interactive line chart
fig = px.line(
    monthly_claims,
    x="Month",
    y="Claim_Amount",
    markers=True
)

# Improve layout
fig.update_layout(
    title=
        "text": "Monthly Claim Trend",
        "x": 0.5,  # center title
        "xanchor": "center",
        "font": {"size": 24, "family": "Arial Black"}
    },
    width=1150,
    height=650,
    xaxis_title="Month",
    yaxis_title="Total Claim Amount",
    hovermode="x unified"
)

fig.show()
```



Q.3) Estimate the loss ratio (Claims ÷ Premium) for the portfolio.

In [27]:
```
total_claims = claims_df["Claim_Amount"].sum()

total_premium = policy_df["Premium"].sum()

loss_ratio = total_claims / total_premium

loss_ratio
```

Out[27]:
```
np.float64(2.049375)
```

Q.4) If claim frequency increases by 5% annually, estimate the impact on future profitability.

In [28]:
```
current_claim_cost = claims_df["Claim_Amount"].sum()

future_claims = current_claim_cost * 1.05
```

```
future_loss_ratio = future_claims / total_premium
future_loss_ratio
```

Out[181]: np.float64(2.15184375)